B7 — PROJETO FISICO E ARMAZENAMENTO

Esta seção explica as decisões de projeto físico do nosso banco. Ela trata de quatro assuntos: se os tipos de dados que escolhemos na Etapa 1 continuam fazendo sentido, quanto o banco deve crescer em dois anos, como seria o plano de backup e se alguma tabela merece ser arquivada ou particionada. Antes de entrar nos detalhes, vale adiantar a conclusão: o nosso banco é pequeno. Estimamos cerca de 110 MB daqui a dois anos, e por isso quase todas as decisões ficaram simples.

1. Revisão dos tipos de dados

Para esta revisão, percorremos coluna por coluna dos scripts 01_ddl.sql , 05_triggers.sql e 06_seguranca_e_controle_de_acesso.sql e nos perguntamos se, com o que aprendemos no semestre, escolheríamos o mesmo tipo hoje. Na maior parte dos casos a resposta foi sim. Encontramos quatro pontos que mudaríamos. Nenhuma dessas mudanças foi aplicada aos scripts entregues; elas ficam registradas aqui como proposta, porque algumas exigiriam alterar a aplicação.

O que mudaríamos:

Separação de CRM e COREN (UF e Número)

Hoje, salvamos o registro em um único texto (ex: CRM-DF 18452), misturando o estado (UF) e o número. Isso dificulta consultas simples no banco de dados (exigindo o uso de LIKE) e quebra a atomicidade exigida pela Primeira Forma Normal (1FN), que determina que cada coluna guarde um único valor.

A proposta:

- Dividir em duas colunas: `crm_numero` (VARCHAR) e `crm_uf` (CHAR).
- Garantir que a combinação de número e estado continue sendo única.
- Aplicar a mesma regra para o COREN dos enfermeiros.

Impacto: Como essa mudança afeta a importação de dados, as telas de cadastro (formulários) e as consultas do sistema, por enquanto a ideia está **apenas registrada** para planejamento futuro.

Telefone do contato de emergência

Hoje, a coluna telefone de Contato_Emergencia é um VARCHAR(20) e faz parte da chave primária. Na nossa carga todos os telefones têm só dígitos (61991110001), mas nada no banco garante isso. Se alguém cadastrasse (61) 99111-0001, o mesmo número entraria duas vezes na chave, como se fossem contatos diferentes.

A proposta:

- Trocar o tipo para VARCHAR(11).
- Adicionar um CHECK que aceite apenas 10 ou 11 dígitos, como já fazemos com o CPF.

Impacto: a regra será aplicada diretamente na estrutura do banco, então a alteração afeta apenas o DDL (criação/estrutura da tabela).

Melhoria nos Logs de Auditoria (Uso de JSON)

Atualmente, as colunas `dados_antigos` e `dados_novos` da tabela `Auditoria_Log` usam o tipo `TEXT` com textos montados por `CONCAT`. Isso dificulta consultas específicas (como buscar alterações de um CPF), pois obriga o uso de `LIKE` em todo o texto.

A proposta:

- Mudar o tipo das colunas para **`JSON`** (nativo do MySQL 8).
- Atualizar os gatilhos (triggers) para estruturar os dados usando **`JSON_OBJECT`** em vez de `CONCAT`.

Impacto: O banco passa a validar o conteúdo e permite buscas estruturadas (utilizando funções como JSON_EXTRACT), tornando as consultas mais precisas.

Padronização do Conjunto de Caracteres do Banco

Atualmente, o script de criação do banco (`CREATE DATABASE`) não define o conjunto de caracteres. Embora o MySQL 8 use `utf8mb4` por padrão, servidores configurados de outra forma podem corromper acentos e caracteres especiais (como em "João").

A proposta:

- Declarar explicitamente `CHARACTER SET utf8mb4` no script de criação.

Impacto: Uma mudança simples de **uma linha** que garante o mesmo comportamento em qualquer servidor, evitando problemas com acentuação.

O que continua como está, e por quê

Tipos de Dados: Chaves, CPF, Dinheiro e Datas

- Chaves e CPF:
  - IDs (`INT AUTO_INCREMENT`): Mantidos como `INT` (limite de 2,1 bilhões), já que o maior volume previsto (`Evolucao_Clinica`, com ~144 mil registros) está muito abaixo disso. Trocar para `BIGINT` traria custo desnecessário de armazenamento e índices.
  - CPF (`CHAR(11)`): Mantido como texto de tamanho fixo para preservar zeros à esquerda (o que se perderia se fosse salvo como número).
- Dinheiro e Datas:
  - Valores em Dinheiro (`DECIMAL(10,2)`**:** Usa tipo exato para evitar erros de arredondamento comuns em tipos `FLOAT`. O teto de R$ 99.999.999,99 é seguro e custa apenas 1 byte a mais que o `DECIMAL(8,2)`.
  - Data e Hora (`DATETIME`): Usado em vez de `TIMESTAMP` para consultas e evoluções clínicas, evitando o "Bug de 2038" e conversões indesejadas de fuso horário.
  - Apenas Datas (`DATE`): Usado para informações como data de nascimento e obtenção de especialidade.

Textos, Senhas e Prescrições

- Textos e Listas Fechadas:
  - Evolução Clínica (`TEXT`): Escolhido porque o tamanho de uma nota médica é imprevisível (evitando truncamentos ou desperdício de limite de um `VARCHAR`). O ponto negativo é que campos `TEXT` não podem ser indexados por inteiro; caso queiram buscar palavras nas notas no futuro, o caminho será usar um índice `FULLTEXT`.
  - Tipo Sanguíneo (VARCHAR(3) com CHECK): São oito valores fixos, listados diretamente na estrutura por meio do CHECK, o que mantém a regra RN10 visível no DDL.
  - Perfil de Usuário (ENUM): Usa um ENUM de três valores, que correspondem aos três usuários do MySQL criados na B5 (recepção, médico e administrador). Se no futuro surgir um quarto perfil, o ideal será migrar para uma tabela Perfil, já que o ENUM exige um ALTER TABLE a cada nova opção.
- Senhas e Prescrições:
  - Senhas (`CHAR(64)` e `CHAR(32)`): O `senha_hash` usa `CHAR(64)` porque o SHA-256 em hexadecimal sempre tem esse tamanho fixo, acompanhado de um

`salt` de 32 caracteres. *Nota futura:* se a aplicação passar a usar bcrypt ou Argon2, a coluna virará `VARCHAR(255)` e o salt separado deixará de ser necessário (pois já vem embutido nesses hashes).

- Prescrições (Dosagem e Frequência): Atualmente são textos livres (ex: `500 mg`, `2 vezes ao dia`), o que impede cálculos automáticos. O ideal seria separar quantidade e unidade, mas como isso exigiria mudanças na aplicação, fica como uma melhoria futura. O limite de `VARCHAR(100)` foi mantido pois não ocupa espaço extra (já que o tipo aloca apenas o que é preenchido).

2. Estimativa de crescimento em dois anos

Para estimar o crescimento da clínica em dois anos, projetamos a operação com base nos dados atuais da nossa carga de testes. Consideramos 14 médicos realizando cerca de 10 consultas diárias em 22 dias úteis, o que resulta em aproximadamente 3.000 consultas mensais. As demais tabelas acompanham essa proporção: calculamos 1,7 prescrições e 2 evoluções clínicas por consulta. Também projetamos a entrada de 150 novos pacientes por mês, cada um com seu prontuário e uma média de 1,1 contato de emergência. Para o log de auditoria, estimamos 400 alterações mensais nas tabelas de pacientes e consultas, já que nossos gatilhos registram apenas atualizações. Por fim, para o tamanho das notas clínicas, adotamos uma média realista de 500 bytes por registro, valor superior aos 80 bytes da carga atual.

Com base nessas premissas, em dois anos a tabela `Evolucao_Clinica` alcançará cerca de 144 mil linhas, `Prescricao` chegará a 122 mil e `Consulta` a 72 mil. A auditoria acumulará perto de 9.600 linhas, enquanto Paciente, Prontuário e Contato de Emergência ficarão entre 3.600 e 4.000 linhas cada. Calculando o espaço (incluindo dados e índices), Evolucao_Clinica consumirá cerca de 86 MB, por ser a única com textos longos. Prescricao ficará em torno de 12 MB, Consulta em 7 MB e Auditoria_Log em 3 MB. As tabelas de cadastro somam menos de 1 MB juntas, e as tabelas de apoio quase não crescem.

Embora seja uma estimativa aproximada, a ordem de grandeza mostra que o banco inteiro cabe quase na totalidade na memória padrão reservada pelo InnoDB (128 MB), o que significa que o volume de dados não exigirá nenhuma estrutura especial a curto prazo. Para acompanhar o crescimento real, a equipe pode rodar a consulta abaixo a cada seis meses e comparar com esta projeção. No InnoDB, o table_rows é apenas uma contagem aproximada de linhas; o tamanho vem da soma de data_length e index_length.

```
SELECT table_name,
       table_rows,
       ROUND((data_length + index_length) / 1024 / 1024, 2) AS
tamanho_mb
FROM information_schema.tables
WHERE table_schema = 'clinica_medica'
ORDER BY (data_length + index_length) DESC;
```

3. Plano de backup

O plano combina um backup completo por dia, feito com mysqldump , e a cópia contínua dos logs binários do MySQL, que registram tudo o que muda no banco. O backup diário nos dá um ponto de partida; os logs permitem recuperar o que aconteceu desde então. Com 110 MB, o mysqldump termina em segundos. As metas que adotamos são perder no máximo 15 minutos de dados e conseguir voltar a funcionar em até uma hora.

O que copiar (Estratégia de Backup)

O backup diário cobre o banco `clinica_medica` inteiro: tabelas, dados, a visão `vw_paciente_recepcao`, a procedure `sp_criar_usuario` e os seis gatilhos (quatro de integridade e dois de auditoria). Os usuários do MySQL (`recepcao`, `medico_app` e `admin_clinica`) não entram nesse dump, então guardamos o arquivo `06_seguranca_e_controle_de_acesso.sql`, que os recria. Os scripts e o código da aplicação já estão no Git. As credenciais da aplicação (`DB_USER`, `DB_PASSWORD` e `DB_URL`) não vão para o Git de propósito, então ficam em um cofre de senhas da equipe.

```
mysqldump -u backup_clinica -p --single-transaction --routines
--triggers --no-tablespaces clinica_medica | gzip >
clinica_$(date +%F).sql.gz
```

A opção --single-transaction faz a leitura do banco inteiro dentro de uma única transação em REPEATABLE READ , o mesmo nível que estudamos na B3. Assim o backup sai consistente, como uma fotografia de um instante, sem travar a aplicação, já que todas as nossas tabelas são InnoDB. O usuário backup_clinica seria um quarto perfil, apenas de leitura (SELECT, SHOW VIEW, TRIGGER e SHOW_ROUTINE), seguindo o princípio do menor privilégio da B5.

Frequência e retenção

O backup completo roda diariamente às 2h da manhã, fora do horário de atendimento, e é mantido por 7 dias. O backup de todo domingo vira o semanal, guardado por 4 semanas, e o do dia 1 de cada mês é o mensal, retido por 12 meses. Os logs binários são copiados a cada 15 minutos e guardados por 14 dias, tempo maior que o intervalo entre dois backups completos. Assim, é possível restaurar o banco até um momento exato antes de um erro (como um apagamento acidental), combinando o último backup com os logs.

Seguimos a regra 3-2-1: três cópias dos dados, em dois tipos de mídia diferentes, sendo uma delas fora da clínica (por exemplo, na nuvem). Todos os arquivos são criptografados, para proteger dados de saúde, que são dados sensíveis segundo a LGPD.

É importante não confundir essa retenção de backups com a guarda dos dados. As cópias duram meses, mas os prontuários precisam ser mantidos por no mínimo 20 anos a partir do último registro (Lei nº 13.787/2018, art. 6º). Por isso as tabelas Prontuario e Evolucao_Clinica nunca são apagadas, e a proposta de arquivamento da seção 4 apenas move dados antigos de lugar, sem eliminá-los.

Como a restauração seria testada

Para garantir que as cópias realmente funcionam, a equipe faria uma simulação uma vez por mês e sempre antes de cada entrega. O procedimento consiste em criar um banco de teste vazio, chamado clinica_teste, e restaurar nele o backup mais recente, para não sobrescrever o banco original:

```
CREATE DATABASE clinica_teste;
gunzip < clinica_2026-10-18.sql.gz | mysql -u root -p clinica_teste
```

Em seguida, a verificação cobre três pontos: se a contagem de linhas de cada tabela é igual à do banco original no momento do backup, se o comando SHOW TRIGGERS lista os seis gatilhos e se as quinze consultas do arquivo 03_consultas.sql retornam os mesmos resultados. Para testar a recuperação até um minuto exato, os logs binários seriam aplicados com a ferramenta mysqlbinlog, usando o parâmetro de data e hora limite. Por fim, cada teste é documentado com a data, o responsável, o tempo gasto (que mede o nosso tempo real de recuperação) e eventuais problemas encontrados.

4. Particionamento e arquivamento histórico

Nenhuma tabela do nosso banco justifica particionamento no horizonte de dois anos. Propomos arquivar apenas a Auditoria_Log e deixar o mesmo método pronto para Consulta e Prescricao , caso um dia precisem. Há dois motivos para não particionar agora. O primeiro é o volume: a maior tabela terá 144 mil linhas, e o particionamento costuma compensar apenas com dezenas de milhões. O segundo é uma limitação do MySQL: uma tabela InnoDB particionada não aceita chave estrangeira. Consulta , Prescricao e Evolucao_Clinica dependem de várias chaves estrangeiras, e abrir mão delas, com seus ON DELETE RESTRICT e CASCADE , quebraria regras de negócio como a RN12 e a RN18.

A tabela `Auditoria_Log` é a candidata natural para o arquivamento porque ela apenas recebe inserções, não possui chaves estrangeiras e quase sempre consultamos apenas os registros recentes. A proposta é manter os dados dos últimos 12 meses na tabela principal e, uma vez por mês, transferir o restante para uma tabela chamada `Auditoria_Log_Arquivo`, onde ficam guardados por 5 anos (um prazo sugerido que pode ser definido pela clínica). Como os dados arquivados continuam no mesmo banco de dados, eles continuam entrando nos backups normalmente. Para garantir a segurança da operação, essa movimentação é feita em uma única transação, evitando que qualquer linha fique duplicada nas duas tabelas ou acabe se perdendo no meio do processo.

```
CREATE TABLE Auditoria_Log_Arquivo LIKE Auditoria_Log;

SET @limite = DATE_SUB(CURDATE(), INTERVAL 12 MONTH);

START TRANSACTION;
INSERT INTO Auditoria_Log_Arquivo
    SELECT * FROM Auditoria_Log WHERE data_hora < @limite;
DELETE FROM Auditoria_Log WHERE data_hora < @limite;
COMMIT;
```

Com cerca de 10 mil linhas, esses comandos não precisam de índice em data_hora. Se a tabela crescer muito, bastaria criar um idx_auditoria_data_hora. Para Consulta e Prescricao , não arquivaríamos nada agora. Quando existirem consultas com mais de 5 anos, o método seria o mesmo, mudando para tabelas de arquivo sem chaves estrangeiras, com uma ordem importante: copiar primeiro as prescrições e só depois as

consultas, porque o ON DELETE CASCADE da chave fk_prescricao_consulta apagaria as prescrições junto com a consulta. Revisaríamos isso uma vez por ano.
A Evolucao_Clinica não seria arquivada nem particionada. O médico precisa do histórico completo do paciente, e o prazo legal de guarda é de 20 anos. A busca por paciente continua rápida graças ao índice que o InnoDB já cria sozinho na chave estrangeira id_prontuario . Se um dia ela ficar lenta mesmo com índice, a alternativa seria particionar por ano, mas isso nos obrigaria a remover as chaves estrangeiras e a garantir a integridade por meio de gatilhos, uma troca que só valeria a pena com um volume muito maior que o previsto. As demais tabelas quase não crescem e não precisam de nenhuma ação.